

UNIVERSIDADE DO MINHO

Licenciatura em Engenharia Informática



Computação Gráfica

2025/2026

Fase 2

Transformações Geométricas

GRUPO 22

106804, Alice Isabel Faria Soares

106853, Ana Beatriz Freitas

107365, Beatriz Martins Miranda

107367, João Paulo Batista Azevedo

Março 2026

Índice

1	Introdução	2
2	Transformações Geométricas	3
2.1	Translação	3
2.2	Rotação	3
2.3	Escala	4
3	<i>Parsing do XML</i>	5
3.1	Transformações Geométricas	5
3.2	Hierarquia de Grupos Aninhados	5
3.3	Estrutura de Dados para Transformações	5
4	Sistema Solar	6
5	Testes	7
6	Conclusão	9

1 Introdução

O presente relatório apresenta as decisões e abordagens adotadas no desenvolvimento da segunda fase do trabalho prático da unidade curricular de Computação Gráfica inserida no 3ºano da licenciatura em Engenharia Informática.

Esta fase consistiu na implementação das transformações geométricas, uma nova funcionalidade do motor gráfico, permitindo a manipulação de objetos através de operações de translação, rotação e escala. Estas transformações são geridas através de uma matriz de transformação, matriz 4×4 em OpenGL/GLUT, que acumula os efeitos sucessivos das operações aplicadas.

Foi também desenvolvida a primeira versão da *scene* do Sistema Solar, servindo como caso de estudo para a aplicação prática destes conceitos. Para suportar esta cena, foram ainda introduzidos os grupos hierárquicos, que permitem organizar os modelos numa estrutura em árvore e facilitar a composição de transformações, possibilitando a criação de sistemas complexos com relações entre objetos.

2 Transformações Geométricas

Foram implementadas três transformações fundamentais no motor gráfico: translação, rotação e escala. Estas operações constituem a base para a construção de *scenes* mais complexas, sendo posteriormente aplicadas na modelação de estruturas hierárquicas, como no caso da *scene* do Sistema Solar.

Um aspeto extremamente relevante na utilização destas transformações é a ordem em que são aplicadas. Uma vez que a composição de transformações é realizada através de multiplicação de matrizes, que é uma operação não comutativa. Desta forma, diferentes ordens de aplicação resultam em comportamentos distintos. Assim, a preservação da sequência de transformações é um aspeto crucial da nossa implementação, garantindo que os objetos apresentam o posicionamento pretendido.

2.1 Translação

A translação permite deslocar um objeto no espaço tridimensional sem alterar a sua forma ou orientação. Esta transformação é definida por um vetor de translação $\vec{t} = (t_x, t_y, t_z)$, que indica o deslocamento a aplicar ao longo de cada um dos eixos cartesianos.

Em termos matriciais, a translação é representada por uma matriz 4×4 , dada por:

$$T(t_x, t_y, t_z) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Quando esta matriz é aplicada a um ponto $\vec{p} = (x, y, z)$, representado por $(x, y, z, 1)$, obtém-se o ponto transformado $\vec{p}'$:

$$\vec{p}' = T(t_x, t_y, t_z) \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ z + t_z \\ 1 \end{pmatrix}$$

2.2 Rotação

A rotação altera a orientação de um objeto no espaço tridimensional, mantendo a sua forma e dimensão. Esta transformação é definida por um ângulo θ e por um eixo de rotação, em torno do qual o objeto é rodado.

Em três dimensões, existem três eixos principais de rotação: os eixos X , Y e Z . Cada um destes eixos possui uma matriz de rotação associada.

- A rotação em torno do eixo X é dada por:

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- A rotação em torno do eixo Y é dada por:

$$R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- A rotação em torno do eixo Z é dada por:

$$R_z(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Tal como nas restantes transformações, estas matrizes são aplicadas aos pontos, permitindo a sua composição com outras transformações através de multiplicação matricial.

2.3 Escala

A escala altera as dimensões de um objeto no espaço tridimensional, modificando o seu tamanho ao longo de cada um dos eixos coordenados. Esta transformação é definida por três fatores de escala (s_x, s_y, s_z), que indicam quanto o objeto é ampliado ou reduzido em cada direção.

Em termos matriciais, a escala é representada por uma matriz 4×4 , dada por:

$$S(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Quando aplicada a um ponto $\vec{p} = (x, y, z)$, representado em coordenadas homogêneas como $(x, y, z, 1)$, obtém-se o ponto transformado $\vec{p}'$:

$$\vec{p}' = S(s_x, s_y, s_z) \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} s_x \cdot x \\ s_y \cdot y \\ s_z \cdot z \\ 1 \end{pmatrix}$$

Verifica-se, assim, que a escala consiste na multiplicação de cada coordenada do ponto pelos respetivos fatores de escala.

Dependendo dos valores utilizados, a escala pode ser:

- **uniforme**, quando $s_x = s_y = s_z$, preservando as proporções do objeto;
- **não uniforme**, quando os fatores diferem entre si, resultando numa deformação do objeto ao longo dos diferentes eixos;

3 Parsing do XML

O *parser* foi estendido para suportar novas funcionalidades, nomeadamente transformações geométricas e hierarquias de grupos aninhados.

3.1 Transformações Geométricas

Foram introduzidos novos elementos XML que permitem definir transformações geométricas dentro de grupos:

```
<translate x="..." y="..." z="..." />
<rotate angle="..." x="..." y="..." z="..." />
<scale x="..." y="..." z="..." />
```

A implementação segue o mesmo paradigma baseado em *callbacks*. Sempre que a função `onStart` deteta uma destas etiquetas, é criada uma instância de `TransformOp`, os seus atributos são preenchidos e a operação é adicionada à lista de transformações do grupo ativo.

3.2 Hierarquia de Grupos Aninhados

Enquanto a Fase 1 suportava apenas um nível de grupos, a Fase 2 permite a definição de grupos arbitrariamente aninhados.

A gestão desta hierarquia é realizada através de uma *stack* com o seguinte comportamento:

- Ao abrir uma etiqueta `<group>`, é criado um novo `Group`, que é adicionado como filho do grupo atualmente no topo da pilha, sendo posteriormente inserido na mesma.
- Ao fechar a etiqueta `</group>`, o grupo no topo da pilha é removido.
- Todas as transformações e modelos são associados ao grupo presente no topo da pilha.
- Caso a pilha esteja vazia, os elementos são associados ao grupo raiz implícito (`scene->root`).

3.3 Estrutura de Dados para Transformações

Para suportar estas funcionalidades, foi introduzida a estrutura `TransformOp` no ficheiro `scene.h`, contendo os seguintes campos:

- `TransformType type`: identifica o tipo de transformação (`TRANSLATE`, `ROTATE`, `SCALE`);
- `float a, b, c, d`: armazenam os parâmetros da transformação (ângulo e eixo, no caso da rotação, ou componentes `x, y, z` para translação e escala).

A estrutura `Group` passa a incluir um vetor `std::vector<TransformOp> transforms`, responsável por armazenar a sequência de transformações associadas a cada grupo.

4 Sistema Solar

Nesta fase, foi desenvolvida uma primeira versão da *scene* do Sistema Solar, com o objetivo de validar o funcionamento das transformações geométricas e da hierarquia de grupos. Esta permitiu testar a aplicação de transformações compostas e a herança entre nós, simulando a relação entre planetas e luas, validando de forma clara os objetivos desta fase. Assim, os planetas são organizados no mesmo nível hierárquico dentro do grupo principal, cada um com as suas próprias transformações (rotação, translação e escala), enquanto as luas são definidas como "filhos" dos respectivos planetas, herdando as transformações do grupo "pai".

Na prática, isto confirma o comportamento esperado da hierarquia de grupos, onde os planetas mantêm a sua transformação independente e, ao mesmo tempo, as luas preservam corretamente a posição relativa ao planeta a que pertencem, demonstrando herança de transformações e composição de matrizes entre nós pai e filho.

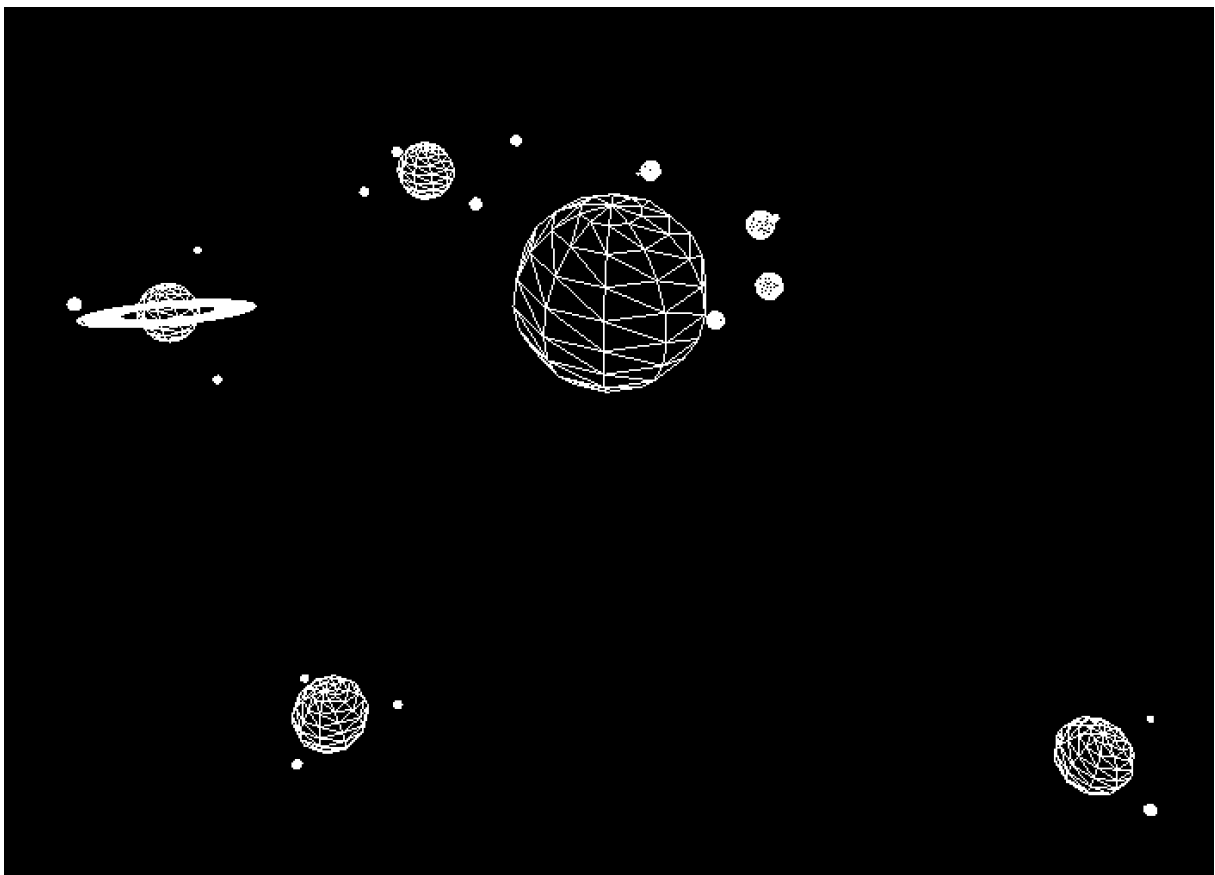
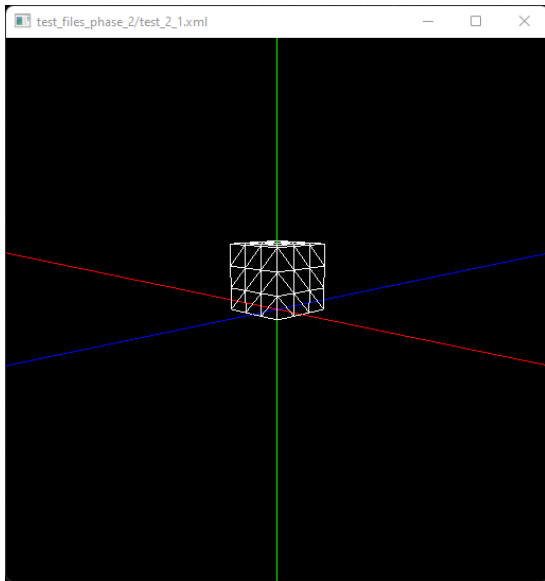


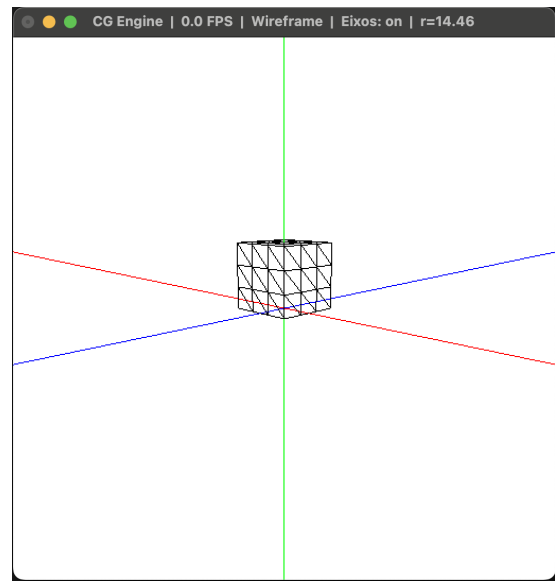
Figura 1: *Scene* sistema Solar

5 Testes

Nesta secção são apresentados os resultados obtidos pelo *engine* para os quatro testes da Fase 2, comparados com os resultados esperados fornecidos pela equipa docente.

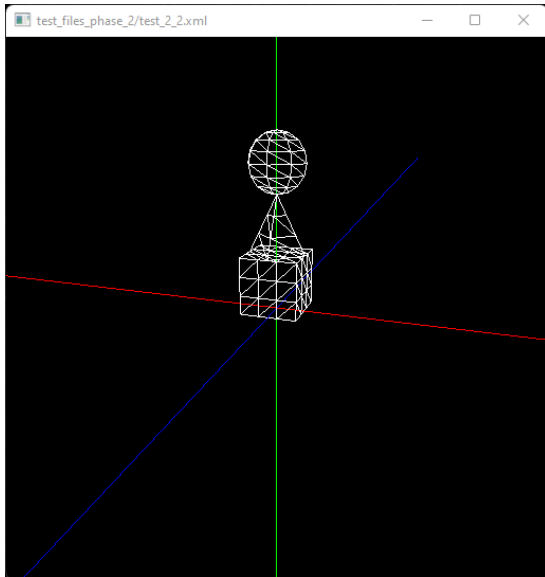


(a) Resultado Esperado

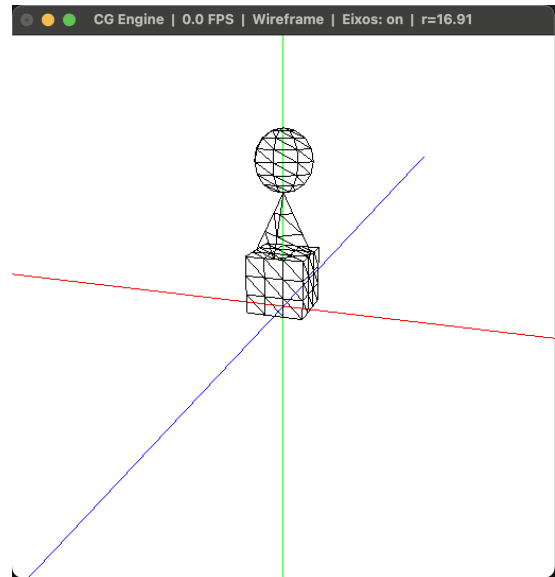


(b) Resultado Obtido

Figura 2: Teste 1

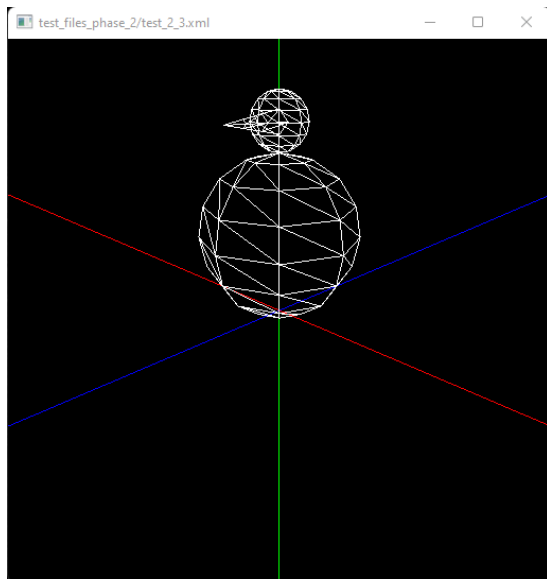


(a) Resultado Esperado

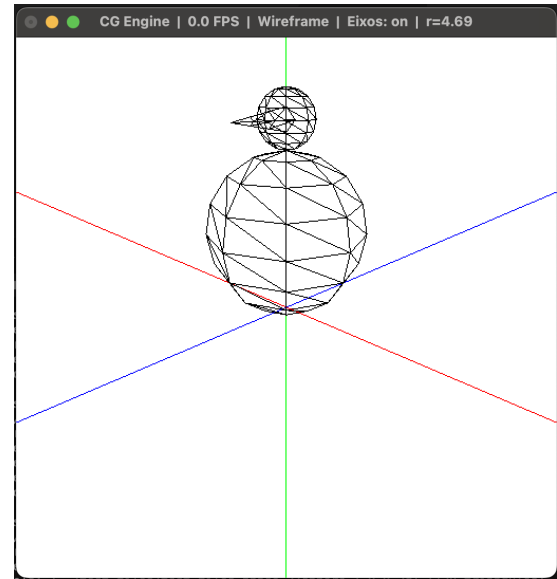


(b) Resultado Obtido

Figura 3: Teste 2

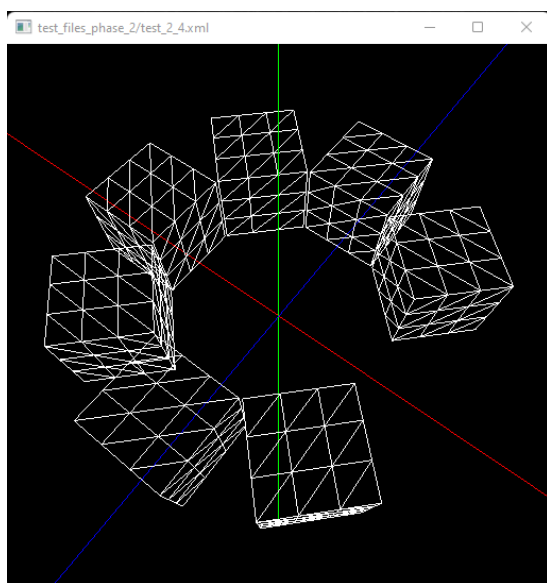


(a) Resultado Esperado

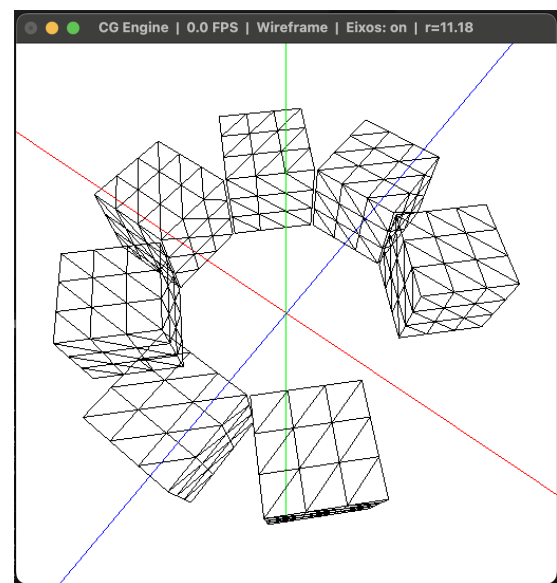


(b) Resultado Obtido

Figura 4: Teste 3



(a) Resultado Esperado



(b) Resultado Obtido

Figura 5: Teste 4

6 Conclusão

A Fase 2 cumpre os objetivos definidos no enunciado, através da implementação de transformações geométricas e do suporte a *scenes* hierárquicas. O *engine* passa a interpretar e aplicar operações de translação, rotação e escala, preservando a ordem de aplicação das mesmas.

Foi introduzido um modelo de grupos em árvore, permitindo a organização de cenas complexas com herança de transformações entre nós, o que possibilita a construção de estruturas hierárquicas como o Sistema Solar.

O parser XML foi estendido de forma consistente com a arquitetura anterior, garantindo compatibilidade retroativa e uma integração modular das novas funcionalidades.

Os resultados obtidos nos testes validam a correta implementação das transformações e da hierarquia de grupos, estabelecendo uma base sólida para as próximas fases do projeto.